

The Antigravity IDE Master Guide

An exhaustive breakdown of every surface, model, modality, tool, and 100% potential execution blueprint.

TABLE OF CONTENTS

- | | |
|---|---|
| 1. The 3 Core AI Modalities | 2. The 7 Exact Models & Reasoning Tiers |
| 3. Chat Canvas, @ Mentions & Slashes | 4. Settings, Policies & Sandboxes |
| 5. Subagents & Background Execution | 6. Core Tools & MCP Server Suites |
| 7. The 3 User Tiers: 0% to 100% Potential | 8. Context Economics & Token Auditing |

1. The 3 Core AI Modalities in Antigravity IDE

Antigravity IDE gives you 3 distinct ways to interact with AI depending on whether you are writing lines, refactoring blocks, or orchestrating systems:

A. Passive: Antigravity Tab (Autocomplete & Supercomplete)

NEXT-INTENT PREDICTION

A single keystroke prediction engine running continuously as you type. Predicts code insertions, multi-line blocks, deletions, and cursor jumps.

- **Tab to Jump:** Anticipates your next navigation point; hit `Tab` to jump straight there.
- **Tab to Import:** Automatically resolves and inserts missing imports at the top of the file.
- **Supercomplete:** Floats larger diffs (including refactorings and deletions) in an overlay.
- **Controls:** Accept with `Tab`, reject with `Esc`, or accept word-by-word with `Ctrl` + `→`.

B. Instructive: Inline Command (`Ctrl` + `I` / `Cmd` + `I`)

LOCALIZED IN-EDITOR EDITS

Surgical in-place code modification without switching context to the chat sidebar.

- **Targeted Refactoring:** Highlight a function or class, press `Ctrl` + `I`, and prompt: "Vectorize with NumPy" or "Add type annotations". The AI modifies ONLY the selected lines.
- **Net-New Generation:** Press `Ctrl` + `I` on a blank line to generate code directly at your cursor.
- **Localized Documentation:** Generates Google/Sphinx docstrings for the selected block instantly.

C. Collaborative: Sidebar Chat & Agent Mode

FULL-STACK PAIR PROGRAMMER

The full agentic engine. Capable of reading/writing files across your workspace, running terminal commands, executing pytest suites, calling MCP tools, and controlling headless browsers.

- **Inline Code Lenses:** Click action buttons floating above functions (e.g., *"Explain Code"*, *"Write Unit Tests"*).
- **Diagnostic Auto-Fix:** Click on compiler/lint squiggly lines or the Problems pane to auto-heal errors directly.
- **Visual Diff Overlays:** In-editor red/green diff review before accepting code changes.

2. The 7 Exact Models & Reasoning Tiers

Decision Rule: Never burn heavy reasoning tokens on simple regexes, and never task a lightweight model with subtle race conditions.

Gemini 3.8 Flash (Medium / Low)

⚡ ULTRA FAST DAILY DRIVER

The Daily Workhorse (80% of all programming tasks). Sub-second latency, highest MCP tool execution speed, lowest token burn, and massive context capacity.

Best For: Writing functions, running pytest, CLI benchmarks, high-volume file edits.

Avoid For: Abstract mathematical architecture with 15+ circular dependencies.

Claude Sonnet 5.5 (Medium)

🧠 GOLD STANDARD UI & CLEAN CODE

Unmatched frontend design aesthetics and idiomatic code taste. Produces stunning, modern web apps (CSS/JS) and pristine, typed Python refactors.

Best For: BioNeMo Cockpit UI, modern web frontends, pristine design patterns.

Avoid For: Repetitive batch grep loops or simple shell test assertions.

Claude Opus 5.5 (Medium)

🧠 MAXIMUM COGNITIVE DEPTH (S-TIER)

Supreme architectural and mathematical reasoning. Solves novel algorithmic challenges, high-stakes system redesigns, and hard bug autopsies.

Best For: Designing core algorithms from scratch, multi-module pipeline graphs.

Avoid For: Fast back-and-forth chat or budget-limited sessions.

Gemini 3.1 Pro (Low)

🧠 DELIBERATE LOGIC & CONCURRENCY

Deliberate multi-hop reasoning. Traces complex concurrency bugs, race conditions, memory leaks, and interface contracts.

Best For: Diagnosing subtle pipeline crashes, security audits, schema boundaries.

Avoid For: Simple one-line edits or trivial renames.

Gemini 3.7 Flash & 3.6 Flash (Medium)

⚡ ROCK-SOLID STABILITY & BATCH

Proven, stable instruction following. Consistent JSON schema adherence without bleeding-edge drift; great for repetitive batch scripts.

GPT-OSS 120B (Medium)

🔗 OPEN-WEIGHT SECOND OPINION

Unbiased open-weights inference. Perfect for independent architectural reviews and cross-model code validation.

Reasoning Modifiers Explained:

- **Low** : Bypasses internal thinking scratchpad. Responds immediately. Saves maximum tokens and time.
- **Medium** : Generates an internal chain-of-thought analysis before outputting code. Prevents logic regressions in complex diffs.

3. Chat Canvas, @ Mentions & Slash Commands

The Power of @ Mentions

CONTEXT INJECTION

Type **@** in the chat input to inject precise context without copy-pasting:

- **@Files & Folders** : Attaches file paths so the agent reads surgical slices.
- **@Previous Conversations** : Bridges context from past debugging sessions.
- **@Terminal Sessions** : Injects terminal output without manual copying.
- **@Rules** : Enforces specific coding guidelines on demand.
- **@MCP Tools** : Explicitly targets a specific MCP tool suite.

Slash Commands (Workflows)

BUILT-IN AUTOMATIONS

Type **/** in chat to trigger specialized agent states:

- **/plan** : Interactive, phased blueprint before touching any code.
- **/goal** : Autonomous endurance mode; runs continuous execution loops until finished.
- **/grill-me** : Interviews you like a Staff Engineer to tease out edge cases.

- `/learn` : Saves corrected rules and workflows into permanent memory across sessions.
- `/schedule` : Configures one-shot timers or recurring cron triggers.

4. Settings, Policies & Sandboxes

Security & Execution Governance

PERMISSIONS

Configured globally or per-project in `.agents/` :

- **Tool Execution Policy:** Choose between `always-proceed` (max autonomy), `request-review`, or `proceed-in-sandbox`.
- **Terminal Sandbox:** Runs shell commands in an isolated container to prevent accidental system changes.
- **Non-Workspace File Access:** Controls whether the agent can read/write outside workspace root (`allow`, `ask`, `deny`).
- **Accidental Data Loss Prevention:** Automatic safety guard blocking destructive commands (e.g. `DROP TABLE`, recursive wipes) without explicit user consent.

5. Subagents & Background Execution

Auxiliary Pane & Reactive Wakeup

NON-BLOCKING FLEET

Antigravity runs a multi-threaded reactive agent architecture:

- **Reactive Wakeup:** When running long jobs or tests, the agent does NOT busy-poll. The system automatically resumes execution when background processes complete.
- **browser_subagent** : Launches an isolated browser that navigates web apps, interacts with buttons/inputs, audits accessibility, and records WebP session videos.
- **Background Task Management:** Control long-running servers and benchmarks using `manage_task` (`list`, `kill`, `status`, `send_input`).

6. Core Tools & MCP Server Suites

- **Core I/O:** `view_file` (line slicing), `replace_file_content`, `multi_replace_file_content`, `write_to_file`, `grep_search`, `run_command`.
- **BioNeMo NIM:** `bionemo_query_diffdock` (docking), `bionemo_query_esmfold` (3D structure), `bionemo_query_esm2_embedding`, `bionemo_sanitise_smiles` (RDKit valence), `bionemo_run_benchmark` (profiler).
- **GitHub MCP:** Full repository management, PR creation, review, and commit pushes.
- **Stitch MCP:** Screen generation from text, UI design systems, and visual variants.

- **Chrome DevTools:** Network inspection, Lighthouse audits, console monitoring, DOM evaluation.
- **Perplexity:** Academic literature and web fact-checking with live citations.

7. The 3 User Tiers: Unlocking 100% Potential

● Level 1: The Rookie (0% – 30% Potential)

GETTING STARTED

Mindset: Transition from passive copy-paster to interactive pair programmer.

1. **Stop Pasting Files:** Never paste 500 lines of code or error logs into chat. Mention the file name with `@`; the agent reads exact line slices.
2. **Adopt Single-Key Tab:** Use `Tab` for auto-imports, insertions, and cursor jumps.
3. **Use `/plan`:** Before any feature, review the agent's interactive blueprint.

● Level 2: The Pro (30% – 70% Potential)

CLOSED-LOOP OPERATOR

Mindset: Eliminate manual verification; automate tests, UI, and background processes.

1. **Closed-Loop Healing:** Always pair edits with terminal tests: "Refactor *X* in `src/engine.py` and run `pytest -v`. Diagnose tracebacks and fix until all pass."
2. **Inline Transforms:** Use `Ctrl + I` for localized refactoring without opening chat.
3. **Multi-Tool Chaining:** Combine BioNeMo docking + RDKit sanitization + GitHub PR in one prompt.
4. **Automated QA:** Deploy `browser_subagent` to verify webapps and record video proof.

● Level 3: The Peak Master (70% – 100% Potential)

FLEET ORCHESTRATOR

Mindset: Act as Mission Director. Shield parent context, match model tiers, and govern multi-agent fleets.

1. **Lead Architect Directive:** The parent agent plans and reviews reports, never consuming bloated raw logs or large source files directly.
2. **Exact Model Matching:** Sonnet 5.5 for UI styling, Opus 5.5 / Pro for deep architecture and race conditions, Flash 3.8 for high-speed execution.
3. **Autonomous Endurance with `/goal`:** Run multi-hour benchmark sweeps or entire test suite refactors without stopping.
4. **Persistent Memory Engineering:** Codify project guidelines into `.agents/rules/` so every session starts at 100% calibration.
5. **Token Economics:** Track exact token burns via `python scripts/token_tracker.py` and eliminate polling waste.

8. Context Economics & Token Auditing

Your Local Token Audit & Tracking Tool

AUDITED ANALYTICS

Antigravity records every turn in `brain/<conv-id>/system_generated/logs/transcript_full.jsonl`.

- Run anytime in terminal: `python scripts/token_tracker.py`
- **Your All-Time Stats:** 13 sessions, 10,695 conversation steps, **3.63M tokens burned**.
- **Input Efficiency:** Only 11.3% prompt input vs 88.7% output generation.
- **Pro Tip:** Use line-bounded slicing (`StartLine` / `EndLine`) to save 85% of input tokens on every turn.